

10강 STL에 대해서 알아보자

Today

- Standard Template Library (STL)

(Summary) STL

- STL 개요
- STL 반복자
- STL 알고리즘
- STL 컨테이너
 - Array
 - Vector
 - ...(다음 강의)

STL

- 템플릿을 사용해 구현된 컨테이너의 집합
 - STL = Standard Template Library
- 널리 사용되는 자료구조와 알고리즘이 구현되어 있음
- 시간 복잡도는 레퍼런스 문서에서 찾아 볼 수 있음
- Powerful, reusable, adaptable, generic, HUGE!

STL

● 구성요소

■ 컨테이너 데이터를 저장하기 위한 객체

- 객체 또는 기본 자료형의 집합
- array, vector, deque, stack, set, map, etc.
- 각 컨테이너는 관련된 헤더 파일 포함 필요 (`#include <컨테이너이름>`)

`#include <array>` 식으로 적어주면 된다

■ 알고리즘

- 컨테이너의 요소들을 다루기 위한 알고리즘
- find, max, count, accumulate, sort, etc.

■ 반복자 (iterators) : 알사용하기 위해 필요하다

- 컨테이너의 요소들에 대한 순회 및 접근
- forward, reverse, by value, by reference, constant, etc.

데이터를 접근할 때 인덱스 대신에 STL에서 구현해놓은 반복자를 사용하기만 하면 된다

STL

● 컨테이너의 종류

▪ Sequential 컨테이너

- 삽입 순서를 유지하는 컨테이너
- array, vector, list, forward_list, deque

▪ Associative 컨테이너

- 순서가 없거나, 미리 정해진 규칙에 따라 저장하는 컨테이너
- set, multi set, map, multi map

▪ 컨테이너 어댑터

- 다른 컨테이너들의 변형 응용
- stack, queue, priority queue

STL

#include <vector>

● 컨테이너의 공통 기능

갖다 쓰되, 어떤 기능(함수)
를 가지고 있는지 알면 됨

Function	Description
Default constructor	Initializes an empty container
Overloaded constructors	Initializes containers with many options
Copy constructor	Initializes a container as a copy of another container
Move constructor	Moves existing container to new container
Destructor	Destroys a container
Copy assignment (operator=)	Copy one container to another
Move assignment (operator=)	Move one container to another
size	Returns the number of elements in the container
empty	Returns boolean – is the container empty?
insert	Insert an element into the container

STL

● 컨테이너의 공통 기능

Function	Description
<code>operator<</code> and <code>operator<=</code>	Returns boolean - compare contents of 2 containers
<code>operator></code> and <code>operator>=</code>	Returns boolean - compare contents of 2 containers
<code>operator==</code> and <code>operator!=</code>	Returns boolean - are the contents of 2 containers equal or not
<code>swap</code>	Swap the elements of 2 containers
<code>erase</code>	Remove element(s) from a container
<code>clear</code>	Remove all elements from a container
<code>begin</code> and <code>end</code>	Returns iterators to first element or end
<code>rbegin</code> and <code>rend</code>	Returns reverse iterators to first element or end
<code>cbegin</code> and <code>cend</code>	Returns constant iterators to first element or end
<code>crbegin</code> and <code>crend</code>	Returns constant reverse iterators to first element or end

STL

- 요소는 복사되어 컨테이너에 저장됨

- 기본 자료형은 모두 OK

포인터 멤버 변수가 있을 때는 복사
생성자와 대입 연산자가 필요하다!

- 사용자 정의 자료형(클래스)은 복사가 가능하고, 대입이 가능해야 함

- 즉, 복사 생성자 / 대입 연산자 필요! ← 포인터 멤버 변수가 없을때는 OK

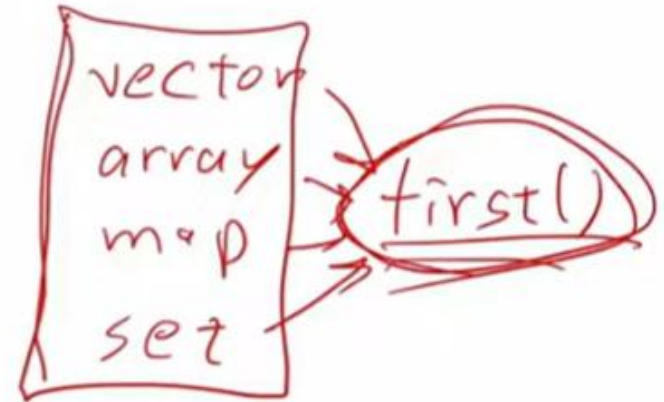
- Associative 컨테이너의 경우, 비교 연산을 수행할 수 있어야 함

- 즉, `operator<`, `operator==` 오버로딩 필요!

STL Iterator

● 반복자(iterator)

- 컨테이너의 요소에 대해 추상적인 접근을 가능하게 하는 기능
 - 추상적 → 컨테이너가 무슨 종류인지, 어떻게 구현되어 있는지 알 필요 없음
- 포인터와 유사하게 사용 가능
- 대부분 컨테이너는 반복자를 사용해 순환(traverse) 가능
 - stack, queue는 예외



컨테이너가 무슨 종류이건 간에, first()를 사용하면 '첫 번째 데이터를 불러온다'(추상적)

STL Iterator

- 반복자의 선언

- 반복자가 사용될 컨테이너의 타입을 명시적으로 표기

컨테이너 타입 컨테이너 내 요소의 자료형

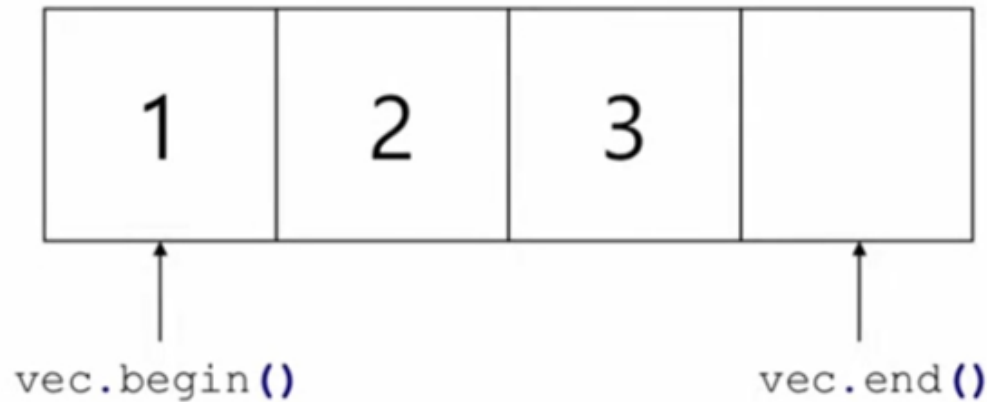


```
vector<int>::iterator it1;  
list<string>::iterator it2;  
map<string, string>::iterator it3;  
set<char>::iterator it4;
```

STL Iterator

- 반복자의 begin과 end

```
vector<int> vec {1,2,3};
```



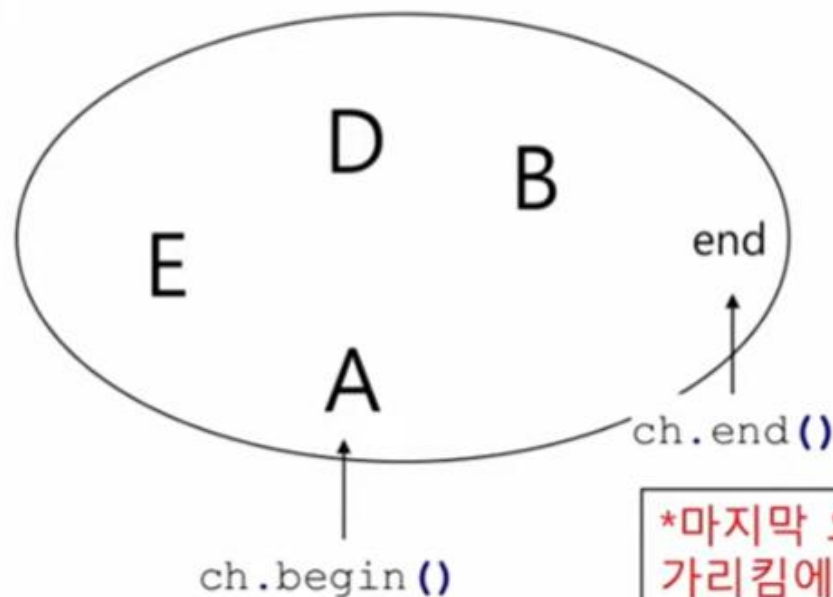
*마지막 요소의 뒤를
가리킴에 주의!

STL Iterator

- 반복자의 begin과 end

- set은 순서가 없음, indexing으로 접근 불가

```
set<char> ch {'A', 'E', 'D', 'B'};
```



*마지막 요소의 뒤를
가리킴에 주의!

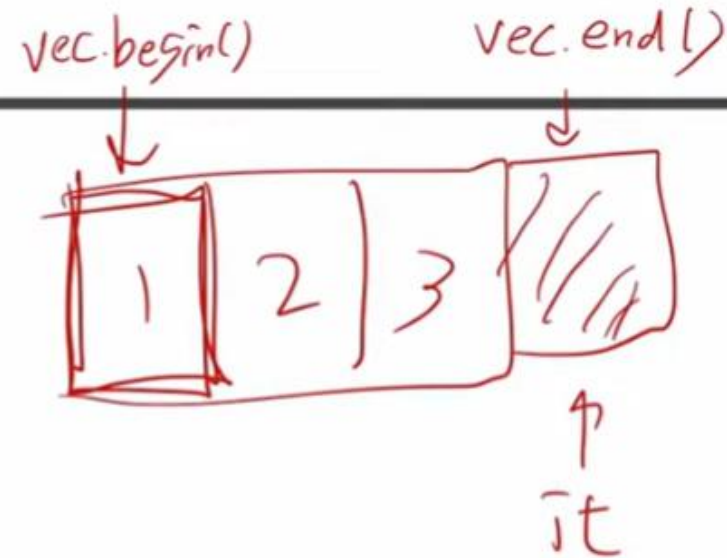
STL Iterator

● 반복자의 사용

- for문과 반복자를 사용한 vector 순환

```
vector<int> vec{1,2,3};
```

```
for(auto it = vec.begin(); it != vec.end(); it++)  
{  
    cout << *it << " ";  
}  
// 1 2 3
```



20페이지 먼저 보고
16페이지로 갑시다

STL Iterator

- 반복자의 초기화

```
vector<int> vec{1,2,3}; //벡터 컨테이너의 선언
```

```
vector<int>::iterator it = vec.begin(); //벡터 컨테이너의 반복자의 선언 및 초기화
```



```
auto it = vec.begin(); //벡터 컨테이너의 반복자의 효율적인 선언 및 초기화 문법!
```

STL Iterator

- Auto 키워드

- 자동으로 타입을 추정해주는 키워드
- 타입이 길어질 경우 사용하면 편리
 - 남용할 경우 코드의 가독성을 해칠 수 있음

```
int Add(int a, int b)
{
    return a + b;
}

int main()
{
    int a = 5;
    auto b = 5; //b is int
    auto c = b; //c is int

    auto d = Add(2, 3); //d is int
}
```

STL Iterator

- 반복자의 연산자

Operation	Description	Type of Iterator
<code>++it</code>	Pre-increment	All
<code>it++</code>	Post-increment	All
<code>it = it1</code>	Assignment	All
<code>*it</code>	Dereference	Input and Output
<code>it-></code>	Arrow operator	Input and Output
<code>it == it1</code>	Comparison for equality	Input
<code>it != it1</code>	Comparison for inequality	Input
<code>--it</code>	Pre-decrement	Bidirectional
<code>it--</code>	Post-decrement	Bidirectional
<code>it + i, it += i</code> <code>it - i, it -= i</code>	Increment and decrement	Random Access
<code>it < it1, it <= it1</code> <code>it > it1, it >= it1</code>	Comparison	Random Access

STL Iterator

- 반복자의 사용

- while문과 반복자를 사용한 vector 순환

```
vector<int> vec{1,2,3};  
  
vector<int>::iterator it = vec.begin();  
  
while(it != vec.end())  
{  
    cout << *it << " ";  
    ++it;  
}  
  
// 1 2 3
```


STL Iterator

- 반복자의 사용

- while문과 반복자를 사용한 set 순환

```
set<char> ch {'A', 'E', 'D', 'B'};

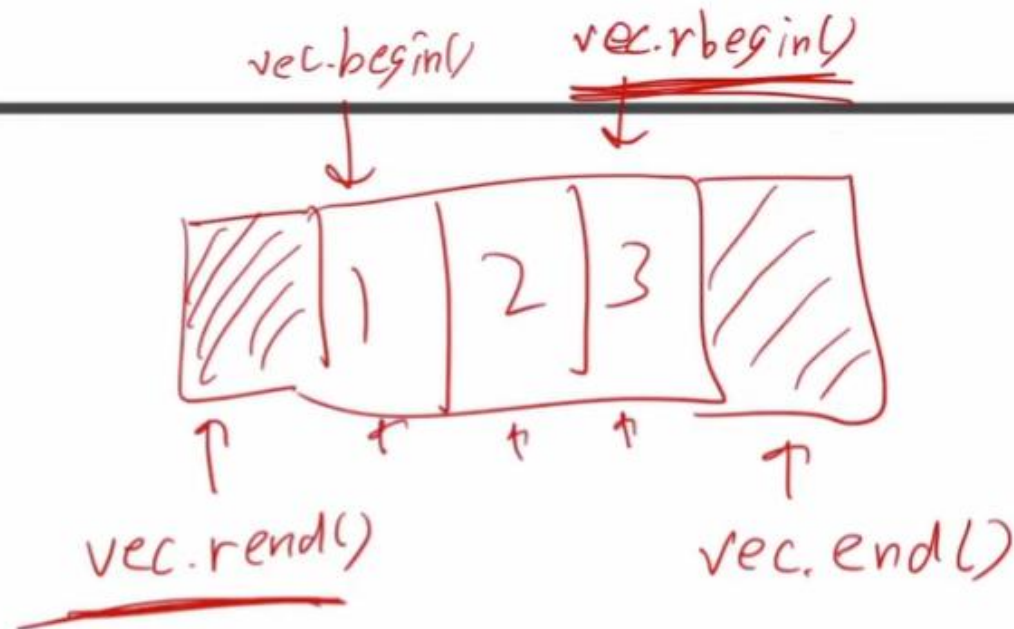
auto it = ch.begin();

while(it != ch.end())
{
    cout << *it << " ";
    ++it;
}
```

STL Iterator

● 역 반복자(reverse iterator)

- 역순으로 동작
- 마지막 요소가 첫 요소가 되는 반복자
- ++ 및 --를 반대 방향으로 생각



```
vector<int> vec{1,2,3};
```

```
vector<int>::reverse_iterator it = vec.rbegin();  
while(it != vec.rend())  
{
```

```
    cout << *it << " ";  
    ++it;
```

```
}  
  
// 3 2 1
```

*iterator 대신 reverse_iterator를 사용함으로써,
동일한 방식으로 반대방향 순환을 수행함

STL Iterator

- 기타 반복자들
 - iterator → begin() / end()
 - const_iterator → cbegin() / cend()
 - reverse_iterator → rbegin() / rend()
 - const_reverse_iterator → crbegin() / crend()

STL Iterator

- 코드 예제
 - 반복자
 - STL Iterator.cpp

STL Algorithms

- STL의 알고리즘

- 반복자에 의해 지정된 요소 집합에 대해 수행되는 알고리즘
- 매우 다양한 알고리즘이 구현되어 있음

- <https://en.cppreference.com/w/cpp/algorithm>

- 알고리즘을 사용하기 위해 함수를 인자로써 제공해야 하는 경우가 존재
- 함수를 인자로써 제공하는 방법

- Functor

- Function pointer

- Lambda expression(C++11)

func(func1)

STL Algorithms

- 알고리즘과 반복자

- `#include <algorithm>`
- 컨테이너에 따라서 적용할 수 있는 알고리즘이 다름
- 모든 알고리즘은 반복자를 인자로 필요로 함

- 반복자의 유효성

- 반복자가 유효하지 않을 수 있음
- 예를들어, iterator가 가리키고 있는 요소가 삭제된다면?
 - iterator가 유효하지 않아짐

STL Algorithms

● 알고리즘 예시 1, find

- find 알고리즘은 컨테이너 내 어떤 요소가 첫 번째로 나타나는 지점을 찾아줌
- 해당 지점을 가리키는 반복자를 반환하거나, end()를 반환함
 - end()를 반환하는 경우, 해당 요소가 컨테이너 내에 없다는 뜻

```
vector<int> vec{1,2,3};  
  
auto loc = find(vec.begin(), vec.end(), 3);  
  
if(loc != vec.end()) // success  
{  
    cout << *loc << endl; // 3  
}
```

STL Algorithms

- 알고리즘 예시 1, find

- find 알고리즘의 사용을 위해서는 요소들을 비교할 수 있어야 함
- operator==이 사용되므로, 클래스의 경우 오버로딩이 필요
 - 오버로딩이 되어있어야만 아래 코드처럼 사용 가능

```
vector<Player> team {...}  
Player p{"Hero", 100, 12};  
  
auto loc = find(team.begin(), team.end(), p);  
  
if(loc != vec.end())  
{  
    cout << *loc << endl;  
}
```

STL Algorithms

- 알고리즘 예시 2, for_each

- for_each 알고리즘은 컨테이너 내 각 요소를 인자로 함수를 호출
- 컨테이너의 각 요소를 제공하는 경우의 예제
- 함수를 인자로 넘기는 방법
 - functor
 - function pointer
 - lambda expression

STL Algorithms

- 알고리즘 예시 2, for_each - Functor의 사용

- 함수(처럼 사용 가능한) 객체

- () 연산자의 오버로딩

```
struct Square_Functor
{
    void operator() (int x) // () 연산자 오버로딩
    {
        cout << x*x << " ";
    }
};
Square_Functor square; //function object
```

Functor의 정의

Functor 객체

```
vector<int> vec {1,2,3,4};
```

```
for_each(vec.begin(), vec.end(), square);
// 1 4 9 16
```

Functor 객체를 인자로 넘겨줌으로써,
Function 안에 구현된 operator() 함수를 각
요소에 대해 호출함

STL Algorithms

● 알고리즘 예시 2, for_each – function pointer의 사용

- 함수의 주소값을 인자로 전달

- square() 와 square의 차이?

- 함수의 호출 / 함수의 주소값

```
void square(int x)
{
    cout << x * x << " ";
}
```

```
vector<int> vec {1,2,3,4};
```

```
for_each(vec.begin(), vec.end(), square);
// 1 4 9 16
```

*아래 코드의 값을 출력해보세요.
cout << square << endl;

← 함수 코드의 주소를 인자로 넘겨줌으로써,
해당 함수를 각 요소에 대해 호출함

STL Algorithms

- 알고리즘 예시 2, for_each – **lambda expression의 사용**

- 익명 함수
- 함수에 이름을 부여하지 않고, 한번 사용하고 버리는(?) 함수 구현에 사용
- 구현 방법은 아래 문서들을 참고
 - [] ← 캡처 절, lambda-introducer (scope 외부 변수를 값으로, 또는 참조로 접근하는지를 명시)
 - <https://en.cppreference.com/w/cpp/language/lambda>
 - <https://modooode.com/196>

```
vector<int> vec {1,2,3,4};
```

```
for_each(vec.begin(), vec.end(),  
        [](int x) {cout << x*x << " "; }) //lambda  
// 1 4 9 16
```


STL Algorithm

● 코드 예제

■ 알고리즘

■ STL_Algorithm.cpp

→ 기능이 구현되어 있는 것. (ex, find, max, for_each...)

→ 어떤 경우 연산자 오버로딩이 되어있어야 함.

P1 == P3

→ 어떤 경우 함수 처리를 인자로 넣어야 함

① functor ② function pointer ③ 익명함수

→ 인자로 변수를 넣는다.

STL Containers

- 컨테이너의 종류

- <https://en.cppreference.com/w/cpp/container>
- Sequence 컨테이너
 - 삽입 순서를 유지하는 컨테이너
 - **array, vector, deque, list, forward_list**
- Associative 컨테이너
 - 순서가 없거나, 미리 정해진 규칙에 따라 저장하는 컨테이너
 - **set, multi set, map, multi map**
- 컨테이너 어댑터
 - 다른 컨테이너들의 변형 응용
 - **stack, queue, priority queue**

Sequence Container - Array

- `std::array` (C++11)
- `#include <array>`
- 고정 크기 배열
- 기존의 정적 배열(raw array) (ex, `int arr[5] = {1,2,3,4,5}`) 과 거의 유사함
- 요소에 대한 직접 접근(constant time)
- STL이므로, iterator와 algorithm을 사용 가능
- 특별한 이유가 없으면, raw array 대신 STL array를 사용

Sequence Container - Array

- Array의 초기화 및 대입

```
array<int,5> arr1 { {1,2,3,4,5} }; //C++11 style  
array<int,5> arr1 {1,2,3,4,5}; //C++14 style
```

```
array<string,3> names{  
    "Kim",  
    "Lee",  
    "Park"  
};
```

```
arr1 = {2,4,6,8,10};
```

Sequence Container - Array

- Array의 주요 함수들 (1)

```
array<int,5> arr {1,2,3,4,5}; //C++14 style
```

```
cout << arr.size(); //5 (배열의 현재 크기 반환)
```

```
cout << arr.at(0); //1 (배열의 0번째 인덱스 값을 반환)
```

```
cout << arr[1]; //2 (배열의 1번째 인덱스 값을 반환)
```

```
cout << arr.front(); //1 (배열의 0번째 인덱스 값을 반환)
```

```
cout << arr.back(); //5 (배열의 마지막 인덱스 값을 반환)
```


Sequence Container - Array

- Array의 주요 함수들 (2)

```
array<int,5> arr {1,2,3,4,5};  
array<int,5> arr1 {10,20,30,40,50};
```

```
cout << arr.empty(); //0(false) (배열에 요소가 있다면 false를, 없다면 true를 반환)  
cout << arr.max_size(); //5 (배열의 최대 크기 반환)
```

```
cout << arr.fill(10); //(배열의 모든 요소를 10으로 채움)  
Arr.swap(arr1); //(두 배열을 swap)  
int *data = arr.data(); //(배열의 주소를 반환)
```


Sequence Container - Array

- 코드 예제
 - array 컨테이너
 - STL_Array.cpp

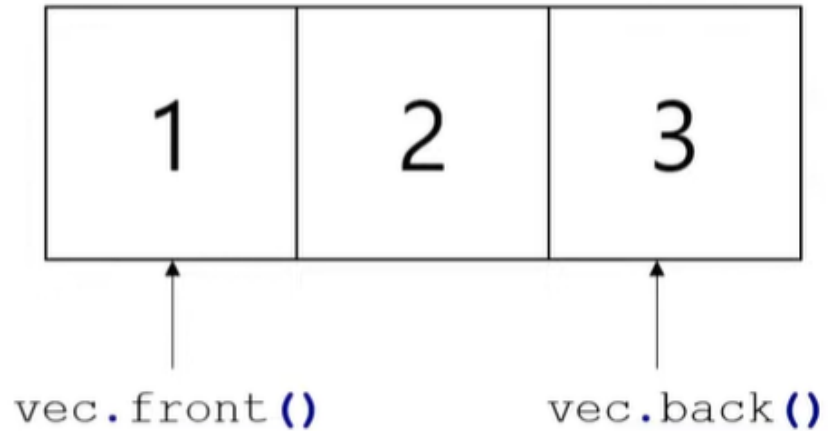
Sequence Container - Vector

- `std::vector`
- `#include <vector>`
- 가변 크기 배열
- 요소에 대한 직접 접근(constant time)
- 요소의 끝 위치 삽입(push_back) 및 제거(pop_back) (constant time)
- 임의 위치에 요소의 삽입 및 제거 (linear time)
- 반복자가 유효하지 않을 수 있음

Sequence Container - Vector

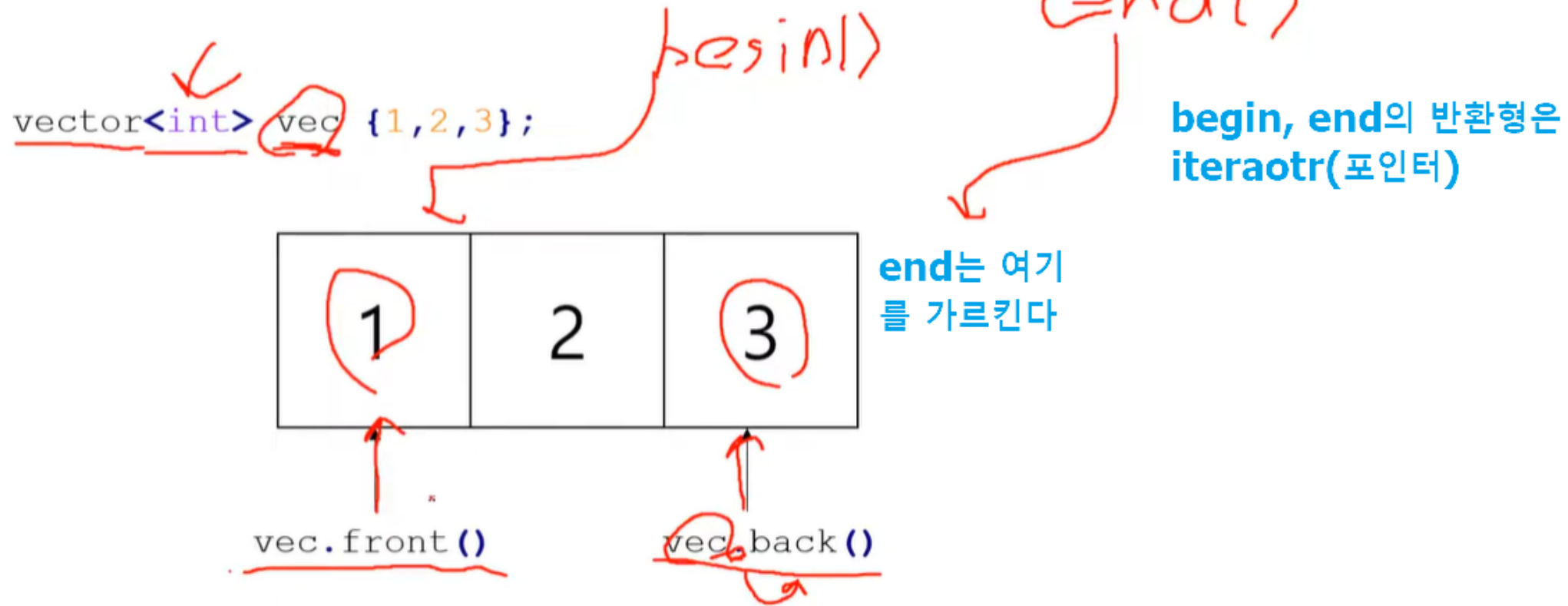
- front / back

```
vector<int> vec {1,2,3};
```



Sequence Container - Vector

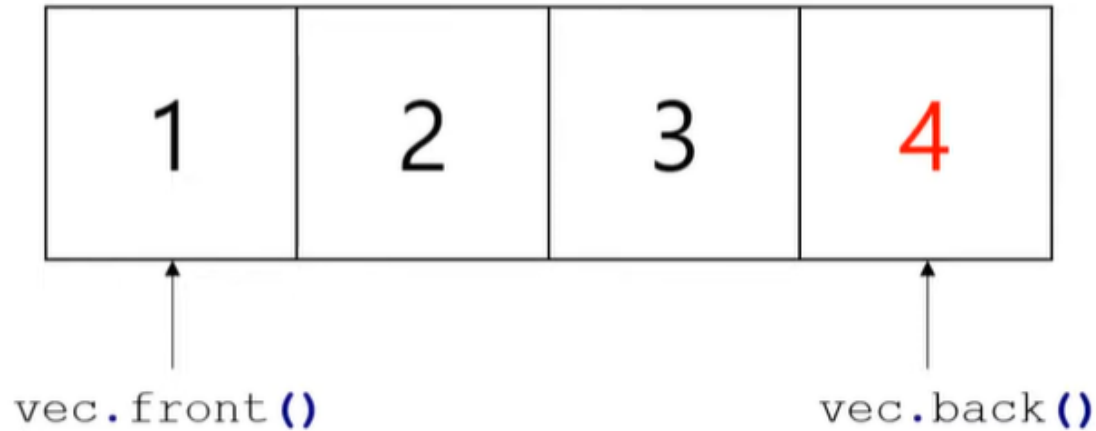
- front / back



Sequence Container - Vector

- front / back

```
vec.push_back(4);
```



Lec10_STL.pptx - PowerPoint

파일 홈 삽입 디자인 전환 애니메이션 슬라이드 쇼 검토 보기 도움말 ACROBAT 스토리보딩 어떤 작업을 원하시나요?

클립보드 슬라이드 글꼴 단락 그리기 편집

42 Sequence Container - Vector

• push / back

43 Sequence Container - Vector

• vector의 초기화 및 대입

```
vector<int> v1 {1,2,3,4,5};
vector<int> v2 {5, 10, 15}; //RHS side
vector<string> v3 {"foo", "bar", "baz"};
v1 = {1,2,3,4,5};
```

44 Sequence Container - Vector

• vector의 주요 함수들 (1)

• max capacity at front back, etc

```
vector<int> v1 {1,2,3,4,5};
cout << v1.size(); //5
cout << v1.capacity(); //5
cout << v1.max_size(); //very large number;
cout << v1.at(1); //2
cout << v1[1]; //2
cout << v1.back(); //5
cout << v1.front(); //1
```

45 Sequence Container - Vector

• vector의 주요 함수들 (2)

• push_back, pop_back, emplace_back

```
Person p1 {"Lee", 33};
vector<int> v1;

v1.push_back(5); // add 5 to the back, copy
v1.pop_back(); // remove 5 from the back

v1.push_back(Person{"Lee", 33});

v1.emplace_back("Lee", 20); //same as above statement, but more efficient!(why?)
//Check how many copy ctor called in push_back
```

46 Sequence Container - Vector

• vector의 주요 함수들 (3)

• empty, swap, etc

```
vector<int> v1 {1,2,3,4,5};
vector<int> v2 {5,10,15,20};

cout << v1.empty(); //false
v1.swap(v2); //swap 2 vectors
```

여기에 슬라이드 노트의 내용을 입력하십시오

슬라이드 45/81 한국어

size = 5
capacity = 5

이 상태에서 `push_back()` 을 하면 더 넣을 공간이 없습

그러면 `vector`는 내부적으로:

1. 더 큰 메모리 공간을 새로 확보함
2. 기존 원소 1,2,3,4,5를 새 공간으로 복사 또는 이동함
3. 새 원소 10을 추가함
4. 기존 메모리 공간을 해제함

이렇게 동작합니다.

이때는:

data 주소가 바뀔 수 있음

입니다.

1. `push_back()` 기본 개념

`<>` C++

```
vec.push_back(Person{"lee"});
```

이 코드는 흐름이 이렇게 됩니다.

Sequence Container - Vector

- vector의 주요 함수들 (3)

- empty, swap, sort

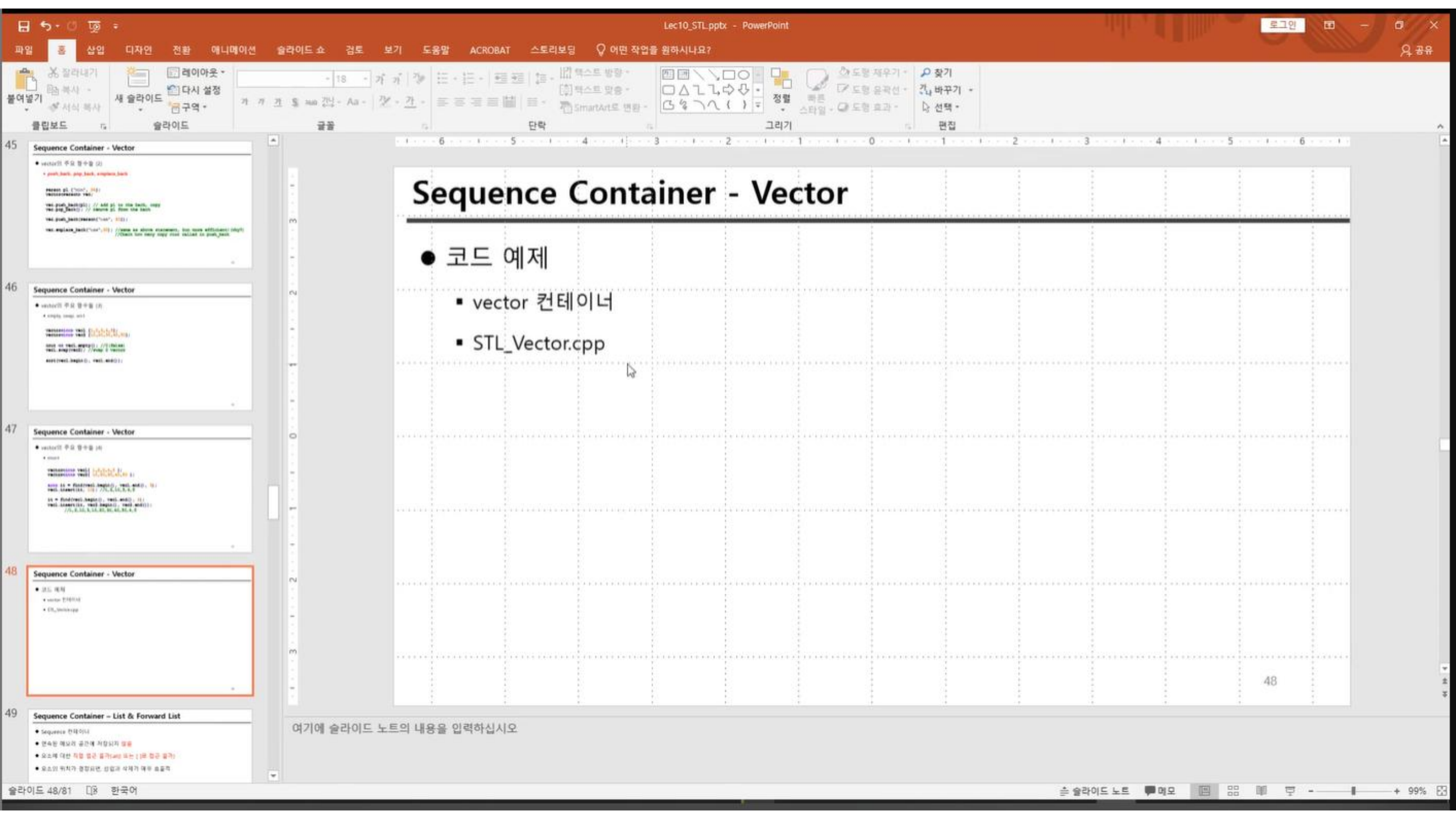
```
vector<int> vec1 {1,2,3,4,5};  
vector<int> vec2 {10,20,30,40,50};  
  
cout << vec1.empty(); //0(false)  
vec1.swap(vec2); //swap 2 vector  
  
sort(vec1.begin(), vec1.end());
```

Sequence Container - Vector

- vector의 주요 함수들 (4)

- insert

```
vector<int> vec1{ 1,2,3,4,5 };  
vector<int> vec2{ 10,20,30,40,50 };  
  
auto it = find(vec1.begin(), vec1.end(), 3);  
vec1.insert(it, 10); //1,2,10,3,4,5  
  
it = find(vec1.begin(), vec1.end(), 4);  
vec1.insert(it, vec2.begin(), vec2.end());  
//1,2,10,3,10,20,30,40,50,4,5
```



Sequence Container - Vector

- 코드 예제

- vector 컨테이너
- STL_Vector.cpp

Sequence Container – List & Forward List

- Sequence 컨테이너
- 연속된 메모리 공간에 저장되지 **않음**
- 요소에 대한 **직접 접근 불가(.at() 또는 []로 접근 불가)**
- 요소의 위치가 결정되면, 삽입과 삭제가 매우 효율적



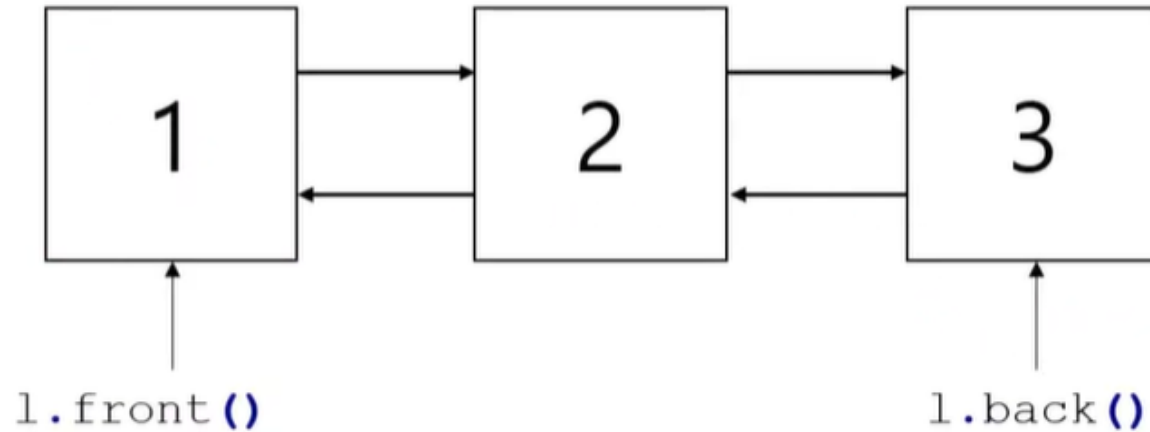
Sequence Container – List & Forward List

- `std::list` (C++11)
- `#include <list>`
- 가변 크기
 - 양방향 연결 리스트
- 요소에 대한 직접 접근 불가
- 임의 위치에 빠른 삽입과 삭제(constant time)
 - 특정 요소를 검색(find)하는 데는 linear time
- 반복자 유효성 보장 안됨

Sequence Container – List & Forward List

- list

```
list<int> l {1,2,3};
```



Sequence Container – List & Forward List

- list의 초기화 및 대입

```
list<int> l1 {1,2,3,4,5};
```

```
list<int> l2 (10, 100);
```

```
list<string> names {"kim", "lee", "park"};
```

```
l1 = {2,4,6,8,10};
```

Sequence Container – List & Forward List

- List의 주요 함수들 (1)

```
list<int> l1 {1,2,3,4,5};
```

```
cout << l1.size(); //5
```

```
cout << l1.max_size; //very large number
```

```
cout << l1.front(); //1
```

```
cout << l1.back(); //5
```

Sequence Container – List & Forward List

- List의 주요 함수들 (2)

```
Person p1{"kim", 20};  
list<Person> l;
```

```
l.push_back(p1); //add p1 to the back  
l.pop_back(); //remove p1 from back
```

```
l.push_front(Person{"lee", 21});  
l.pop_front(); //remove front element
```

```
l.emplace_back("park", 22); //efficient add to back  
l.emplace_front("choi", 25); //efficient add to front
```

Sequence Container – List & Forward List

- List의 주요 함수들 (3)

```
list<int> l1{1,2,3,4,5};
```

```
auto it = find(l1.begin(), l1.end(), 3);
```

```
l1.insert(it,10); //1 2 10 3 4 5
```

```
l1.erase(it); //1 2 10 4 5 ←———— It 유효성 사라짐
```

```
l1.resize(2); //1 2
```

```
l1.resize(5); //1 2 0 0 0
```

Sequence Container – List & Forward List

- List의 주요 함수들 (4)

```
list<int> l1{1,2,3,4,5};  
  
auto it = find(l1.begin(), l1.end(), 3);  
  
cout << *it; //3  
it++;  
cout << *it; //4  
it--;  
cout << *it; //3
```

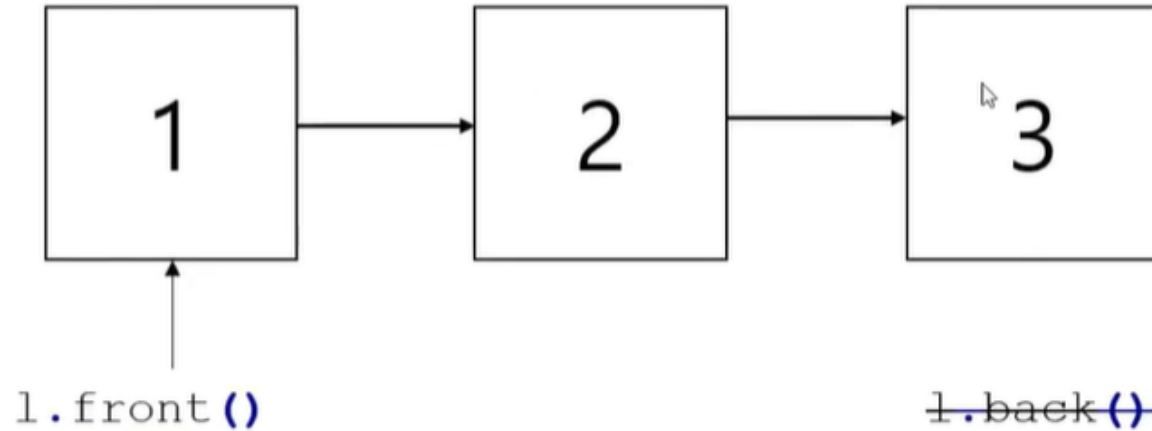
Sequence Container – List & Forward List

- `std::forward_list` (C++11)
- `#include <forward_list>`
- 가변 크기
 - 단방향 연결 리스트 (메모리 절약)
- 요소에 대한 직접 접근 불가
- 임의 위치에 빠른 삽입과 삭제(constant time)
 - 특정 요소를 검색(find)하는 데는 linear time
- 반복자 유효성 보장 안됨
- reverse iterator 사용 불가

Sequence Container – List & Forward List

- forward_list

```
forward_list<int> l {1,2,3};
```



Sequence Container – List & Forward List

- 코드 예제
 - List, forward_list 컨테이너
 - STL_List.cpp

Associative Container – Set

- Associative 컨테이너

- Key를 사용한 빠른 데이터 검색이 가능
- 주로 Balanced binary tree 또는 hashset으로 구현됨
- 대부분의 연산이 빠르게 이루어짐

- Set

- Set
- Unordered_set
- Multiset
- Unordered_multiset

Associative Sequence Container - Set

- `std::set`
- `#include <set>`
- 집합과 유사(중복된 요소 없음)
- key에 따라 정렬
- 요소에 대한 직접 접근 불가
- 반복자가 유효하지 않을 수 있음

2. key에 따라 정렬한다는 말

`set<int>`에서는 값 자체가 key입니다.

`</> C++`

```
std::vector<int> v{ 3, 1, 2, 3, 2 };
```

이러면 실제로 이렇게 저장됩니다.

3 1 2 3 2

그런데 `set`은 같은 값을 여러 번 넣어도 **하나만 남습니다.**

1
2
3

3을 두 번 넣고, 2도 두 번 넣었지만 결과에는 하나씩만 남습니다.

즉:

`set` = 같은 값은 중복 저장하지 않음

입니다.

~~Associative~~ Sequence Container - Set

- set의 초기화 및 대입

```
set<int> s {1,2,3,4,5};
```

```
set<string> names {"kim", "lee", "park"};
```

```
s = {2,4,6,8,10};
```

Associative Sequence Container - Set

- set의 주요 함수들 (1)

```
set<int> s {4,1,1,3,3,2,5}; //1 2 3 4 5
```

```
cout << s.size(); //5
```

```
cout << s.max_size; //very large number
```

```
//no front or back
```

```
s.insert(7); //1 2 3 4 5 7
```

Associative Sequence Container - Set

- set의 주요 함수들 (2)

```
Person p1 {"kim", 20};  
Person p2 {"lee", 21};
```

```
set<Person> names;
```

```
names.insert(p1); //add p1 to the set  
auto result = names.insert(p2); //add p2 to the set
```

- 정렬을 위해 operator< 오버로딩 필요!

- insert의 결과로 pair<iterator,bool> 반환

- Iterator는 삽입된 위치, 또는 이미 존재하는 요소를 가리키는 반복자 반환
- Bool은 성공 또는 실패 결과 반환

~~Associative~~ Sequence Container - Set

- set의 주요 함수들 (3)

```
set<int> s {1,2,3,4,5};
```

```
s.erase(3); //1 2 4 5
```

```
auto it = s.find(5);  
if(it != s.end())  
    s.erase(it); //1 2 4
```

← Algorithm의 find 함수가 아닌, set의 멤버 변수인 find

Associative Sequence Container - Set

- set의 주요 함수들 (4)

```
set<int> s {1,2,3,4,5};
```

```
int num = s.count(1); //0 or 1
```

```
s.clear(); //remove all elements
```

```
s.empty(); //true or false
```

~~Sequence~~ Container - Set

~~ASOC.~~

- multiset

- Key로 정렬
- 요소의 중복 허용

- unordered_set

- 요소의 중복 불가
- 요소가 정렬되지 않음
- 요소의 수정 불가

- unordered_multiset

- 요소가 정렬되지 않음
- 요소의 중복 허용

set 1 2 3 4 5
3 1 4 2 5 2
m.set 1 2 2 3 4 5
u.set 3 1 4 2 5
u.m.set 3 1 4 2 5 2

Sequence Container – Set

- 코드 예제

- STL_Set.cpp

Associative Container – Map

- Associative 컨테이너

- Key를 사용한 빠른 데이터 검색이 가능
- 주로 Balanced binary tree 또는 hashset으로 구현됨
- 대부분의 연산이 빠르게 이루어짐

- Map

- Map
- Unordered_map
- Multimap
- Unordered_multimap

Sequence Container – Map

- `std::map`
- `#include <map>`
- Dictionary(사전)과 유사
- 요소가 key-value 쌍으로 저장됨
- Key 값에 따라 정렬
- Key는 유일한 값으로 중복 불가
- Key를 통한 직접 접근 가능
- 반복자가 유효하지 않을 수 있음

네. 여기서 **Dictionary(사전)**은 “단어를 찾으면 뜻이 나오는 구조”라고 생각하시면 됩니다.

예를 들어 실제 사전은 이렇게 생겼습니다.

```
apple  → 사과  
banana → 바나나  
cat    → 고양이
```

여기서:

```
apple  = key  
사과   = value
```

입니다.

C++의 `std::map`도 똑같이 **key-value 쌍**으로 데이터를 저장합니다.

Sequence Container – Map

- map의 초기화 및 대입

```
map<string,int> m1 {  
    {"kim", 20},  
    {"lee", 21}  
};  
  
map<string,string> m2 {  
    {"kim", "student"},  
    {"lee", "professor"}  
};
```


Sequence Container – Map

- map의 주요 함수들(2)

```
map<string,string> m2 {  
    {"kim", "student"},  
    {"lee", "professor"}  
};
```

```
pair<string, string> p1 {"park", "assistant"};
```

```
m2.insert(p1);
```

```
m2.insert(make_pair("choi", "employee"));
```

Sequence Container – Map

- map의 주요 함수들(3)

```
map<string,string> m2 {  
    {"kim", "student"},  
    {"lee", "professor"}  
};  
  
m2["park"] = "assistant"; //insert  
  
m2["park"] = "employee"; //update value  
m2.at("park") = "student"; //update value
```

Sequence Container – Map

- map의 주요 함수들(4)

```
map<string,string> m2 {  
    {"kim", "student"},  
    {"lee", "professor"}  
};  
  
m2.erase("kim");  
  
if(m2.find("lee") != m2.end())  
    cout << "fount lee!";  
  
auto it = m2.find("lee");  
if(it != m2.end())  
    m.erase(it);
```

Sequence Container – Map

- map의 주요 함수들(5)

```
map<string,string> m2 {  
    {"kim", "student"},  
    {"lee", "professor"}  
};  
  
int num = m2.count("kim"); //0 or 1  
m2.clear(); //remove all elements  
m2.empty(); //true or false
```

Sequence Container – Map

- multimap

- `#include <map>`에 포함되어 있음
- 중복된 요소를 허용

- unordered map

- `#include <unordered_map>`
- 요소가 정렬되지 않음

- unordered multimap

- `#include <unordered_map>`에 포함
- 요소가 정렬되지 않음
- 중복된 요소를 허용

Sequence Container - Map

- 코드 예제
 - STL_Map.cpp

- 구성요소

- 컨테이너

- 객체 또는 기본 자료형의 집합
 - array, vector, deque, stack, set, map, etc.
 - 각 컨테이너는 관련된 헤더 파일 포함 필요 (#include <컨테이너이름>)

- 알고리즘

- 컨테이너의 요소들을 다루기 위한 알고리즘
 - find, max, count, accumulate, sort, etc.

- 반복자 (iterators)

- 컨테이너의 요소들에 대한 순회 및 접근
 - forward, reverse, by value, by reference, constant, etc.

